

Argus Security Analysis Report

Repository: https://github.com/arnav-sutraway/Vuln_test

Total Findings: 11

1. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 88.0%

Location: admin.py, line 55

Evidence: query = f"SELECT * FROM audit_logs WHERE action LIKE '%{search_term}%'"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

2. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: auth.py, line 22

Evidence: `f"WHERE username = '{username}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

3. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: auth.py, line 39

Evidence: + "" WHERE email = ""

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

4. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 11

Evidence: query = "SELECT * FROM users WHERE username = " + username + ""

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

5. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 24

Evidence: query = f"SELECT * FROM users WHERE id = {user_id}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

6. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 39

Evidence: `f"WHERE username = '{username}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

7. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 74

Evidence: query = f"SELECT * FROM products ORDER BY {sort_column}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

8. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services\reports.py, line 10

Evidence: `f"WHERE type = '{report_type}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

9. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services\reports.py, line 24

Evidence: query = f"SELECT * FROM {table_name}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

10. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services\users.py, line 10

Evidence: "WHERE email = " + email + " "

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

11. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services\users.py, line 24

Evidence: query = f"DELETE FROM users WHERE id = {user_id}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.